

Hunting Mr. X: Wellington Edition

Ming Bao

Abstract—

- **Problem:** hidden-movement board game limited to co-located play; existing digital versions use fictional boards and no real geography.
- **Solution:** browser-based, real-time multiplayer game on a 216-node, 372-edge graph of Wellington's bus, train, ferry, and e-scooter network.
- **Method:** Java/Spring Boot authoritative server, Vue 3 + MapLibre GL client, STOMP over WebSocket, role-filtered state so Mr X's position is never sent to detectives.
- **Evaluation:** automated tests (134 backend tests across unit, mock, lifecycle, integration, functional and property-based/fuzz layers, plus an opt-in latency test; 99.2% line and 97.8% branch coverage; 100% mutation score), map-library benchmark, two user-testing rounds, System Usability Scale (SUS) questionnaire (overall 72.5, $n = 7$, against the 68 average benchmark).
- **Outcome:** deployed, playable system suitable for Open Day demonstration.

Index Terms—Multiplayer game, hidden movement, real-time web application, WebSocket, map overlay, usability evaluation.

I. INTRODUCTION

- Motivation: board games tied to physical components and co-location; hidden-movement deduction games translate well to networked play because the server can enforce secrecy.
- Gap: existing digital versions replicate the original board; no browser-based version on a real city map with real transport modes.
- Secondary goal: showpiece for university Open Day (visually compelling, robust enough for a general audience).
- Aim and quantifiable goals (state early, refer back in Evaluation):
 - Automatic rule enforcement (movement, tickets, reveal rounds, double move, win conditions, 24 rounds).
 - Map render time under 3 s for roughly 200 nodes / 285 edges (spec target).
 - State push under 500 ms from move submission to all clients (spec target).
 - Multiple concurrent games; disconnect handling with a grace period.
 - Usability: SUS above the 68 benchmark.
- Contributions: static pre-computed Wellington graph and map-creation tooling; role-filtered real-time architecture; evaluation combining benchmarks, automated tests, and user testing.
- Naming: game mechanics based on the Ravensburger board game, used with permission for non-commercial academic purposes; the project was proposed under a

working title and renamed, so the original brand name is not used.

- Key findings (one line each, from Evaluation).
- Report roadmap (one sentence per section).

II. RELATED WORK

- Game rules being implemented: hidden movement, transport tickets, reveal rounds, double-move and invisible tickets (cite the Ravensburger rulebook).
- Existing digital versions (Board Game Arena, Steam ports): strong rules, fictional map, no real geography. Find and cite specifics; the progress-presentation table is a starting point.
- Location-based games (Pokémon GO and similar): real geography but shallow strategy; comparison table (real map / browser / strategic depth / real-time multiplayer).
- Hidden-information multiplayer design: authoritative server, never sending secret state to clients (cheat resistance); cite a game-networking source.
- Map libraries: Google Maps, Leaflet, MapLibre GL. Cite documentation and licences [1].
- Routing approach: routing Application Programming Interfaces (APIs) (per-request cost and latency) versus pre-computed GeoJSON; OpenStreetMap as data source (cite licence/attribution).
- Real-time web transport: WebSocket, Simple Text Oriented Messaging Protocol (STOMP), SockJS fallback versus polling or server-sent events.
- Usability evaluation: SUS (Brooke; Sauro's 68 average benchmark).
- Critical analysis: strengths, weaknesses, and gaps of each; close with how this project addresses the gap.

III. REQUIREMENTS AND CONSTRAINTS

- Functional requirements (from `documentation/spec.md`): roles (1 Mr X, 1–5 detectives), four transport modes, ticket allocation, turn order, reveal rounds, double move, Invisible ticket, win conditions, movement constraints.
- Non-functional requirements: latency, render time, concurrency, browser support (latest Chrome/Firefox/Safari), desktop-first.
- Constraints: no per-request paid APIs; in-memory only (no database); solo project over 24 weeks; budget \$294 (from proposal); self-hosted deployment.
- Table: requirement, measure, target, where evaluated (traceability from requirements to the Evaluation section).

IV. DESIGN

A. Architecture

- Client–server, authoritative server; figure: browser (Vue + Pinia + MapLibre) / REST / STOMP / Spring Boot / in-memory `ConcurrentHashMap`.
- Why REST for actions and WebSocket for push only (simpler auth and error handling).
- Alternatives considered and rejected: peer-to-peer, polling, database-backed sessions, Node backend (Java vs JavaScript from the proposal); trade-off for each.

B. Game State Machine and Turn Flow

- Phases: Lobby, InProgress (MrXTurn, DetectiveTurn, RoundEnd), ended states (detectives win, Mr X wins, aborted); the lobby closes if the host leaves (figure from `plans/states-diagrams.md`).
- Every operation validated against phase (e.g. start only in lobby, kick only in lobby).
- Leaving and idling: any player leaving a game in progress aborts it, and the server aborts a game after 15 minutes without a move. Pause-on-disconnect with a 60 s grace period was planned but dropped: user-testing networks dropped WebSockets often, so clients reconnect and re-sync over REST instead of the server pausing.

C. Domain Model

- Player model: one `Player` class whose role is assigned at start; every rule lives in a plain `Game` class with no Spring, which made the rules directly unit-testable (the earlier lobby/Mr X/detective class hierarchy and abstract player were collapsed in the backend rewrite).
- Ticket model: unlimited regular tickets for Mr X, finite Invisible and Double; fixed budgets for detectives.
- Class diagram (from `plans/classdiagram.md`).

D. Hidden Information and Role-Filtered State

- Each player receives a filtered snapshot; Mr X's node omitted on non-reveal rounds, reveal appears in log.
- Per-player topics rather than one shared topic (explain why).
- Sequence diagram of a move: submit, validate, apply, catch check, broadcast per-player views.

E. Map Data Design

- Decision: static graph JSON with inline GeoJSON geometry versus live routing APIs (cost, latency, hundreds of edges).
- Node and edge format, undirected edges, mode arrays; graph density and balance between the four modes.
- Library selection: benchmark methodology and results (see Evaluation).

F. User Interface Design

- Screens: landing, create, join, lobby, game board, end screen; dark theme, mode colours, ticket selector, info panel, Mr X log with reveal rows.
- Design decisions tied to feedback (see Evaluation): transport-mode popups, viewport-relative layout, centred turn indicator.
- Mockups or Figma reference (if used).

G. Real-World Issues

- Ethics: voluntary student participants, course coordinator approval; vouchers for participants (check the ethics wording).
- Sustainability: no per-request cloud calls, minimal self-hosted infrastructure.
- Safety and IP: no location tracking of real players; Ravensburger permission; open-source licences (GPL-3.0 for repo; map data attribution).
- Privacy: what is stored (in memory only), no accounts.

V. IMPLEMENTATION

- Technology table with justification: Java 21, Spring Boot 4.0.6 (`spec.md` says 3, verify), Maven; Vue 3, Pinia, Vue Router, TypeScript, Vite, Tailwind CSS 4; MapLibre GL 4; `@stomp/stompjs` and `SockJS`; Docker, nginx, supervisord, Cloudflare Tunnel.
- Backend structure: controllers (`GameController`, `MapController`, `ApiExceptionHandler`), `GameService`, `MapGraph`, models, records for Data Transfer Objects (DTOs) (about 1,200 lines of Java after simplification pass).
- Move validation: edge lookup, ticket check, occupied-node rule, catch detection, auto-skip of stuck detectives; per-game locking for concurrency.
- Reveal rounds (2, 8, 13, 18, 24) implemented once in the backend and mirrored in a frontend utility.
- Double move and Invisible ticket handling; log entries for each leg.
- Configuration properties: grace period, turn timer, detective ticket budgets (12 e-scooter / 8 bus / 6 train / 2 ferry).
- Frontend: Pinia store as single source of truth, STOMP subscriptions, REST fallback with polling for reconnect (6 s), MapLibre layers per mode, reachable-node highlighting, node popups.
- Map creation: `mapCreator` tool (browser tool + headless generator/evaluator); building from bus and train networks first, then e-scooter edges; problems: fully connected graphs, awkward road geometry, hard to create three-colour side-by-side edges (from notes).
- Final graph statistics: 216 nodes, 372 edges (323 e-scooter, 257 bus, 9 train, 2 ferry edge-mode incidences).
- Deployment: single Docker image (nginx + Spring Boot under supervisord), `BASE_PATH` support, update script, Cloudflare Tunnel on a home server; live at the deployed domain.

- API documentation: OpenAPI spec (currently 0.1.11) kept in sync with REST and STOMP topics, enforced in the workflow.
- Development process: iterative sprints; use of AI-assisted development (Claude Code) under review and ownership; describe what it was used for and how output was verified. Check how the course expects AI use to be disclosed.
- Figures: screenshots of lobby, board, ticket selector, log, end screen.

VI. EVALUATION

A. Map Library Benchmark

- Method: Selenium + Firefox, 10 runs each, baseline and stress scenarios (`time_maps.py`, `scenarios.json`).
- Results table (mean, s): baseline Google 0.70 / Leaflet 0.23 / MapLibre 0.40; stress 2.69 / 2.21 / 2.31.
- Discussion: Leaflet fastest but MapLibre gives WebGL and lowest variance in stress; Google needs a key and cost; MapLibre chosen. Note that the benchmark measured empty-map and synthetic load, not the final graph.
- Threats: single browser, single machine, small run count.

B. Functional Correctness

- Test suite: 134 backend tests: unit (82: MapGraph, Player, Game on hand-drawn boards), mock (17: GameService with mocked messaging), lifecycle (8 scripted complete games), integration (19: HTTP via MockMvc, STOMP with real clients), functional (4: full games over HTTP and STOMP), property-based and fuzz (4, JUnit with seeded random), and performance (1, opt-in); mapped to the proposal's plan.
- Table of layers, counts and what each catches (counts and results in `documentation/backend-fixes.md`).
- Targeted tests: every bug found in the backend review has a regression test (`documentation/backend-fixes.md`), e.g. Mr X boxed in, chained double moves, hidden position not leaked over REST or STOMP.
- Full-game end-to-end test through the real server with an HTTP and a STOMP client for every player (it replaced a Selenium test that only used the browser as an HTTP client and was flaky).
- Meta-testing: JaCoCo coverage 99.2% lines and 97.8% branches overall (game and service: 99.7% and 97.7%); PIT mutation score 100% (201 of 201 mutants killed) on game and service, after the first run's survivors exposed seven missing tests; property-based tests over 300 seeded random games (every rule holds and every game ends) and fuzz tests (garbage moves, names and request bodies never cause a server error). Frontend has no automated tests.

C. Performance and Multiplayer Synchronisation

- Targets: 500 ms push latency, 3 s map render, concurrent games.

- Latency harness (`MultiplayerPerfTest`): 10 concurrent games of 6 players on the real map (987 moves); move submitted to every other player updated p50 4.6 ms, p95 8.3 ms, max 36.6 ms, against the 500 ms target. A render-time measurement with the real map is still to do.
- Report percentile latencies as a figure/table; compare against targets.

D. User Evaluation

- Participants: seven players (six detectives, one Mr X), games of four to six players; voluntary, ethics approved, voucher incentive.
- Instrument: SUS plus free-text comments and role / player count / perceived side balance.
- Results table: Mr X 70.0 (n = 1), detectives 72.9 (n = 6), overall 72.5 (n = 7); above the 68 benchmark; range 60–95.
- Qualitative themes: onboarding and tutorial demand (three responses), Mr X log hard to read at first, reachable-node scaling on small screens, praise for fun and replayability, request for custom lobby rules.
- Iteration evidence: test round one (seven issues) and round two (four issues) mapped to the fixes (name limit, zoom limit, tall log, node popups, final-state view, colour diversity, selection highlight bug, centred turn indicator, reveal rows in log, unabbreviated rounds).
- Limitations: small n, one Mr X respondent, convenience sample of students, SUS tested after fixes were partly applied. Possibly add before/after comparison if a third round is run.
- Side balance: column “which side felt stronger” was answered 2 by everyone; decide how to interpret and report (check the form's scale).

E. Requirements Traceability

- Table mapping each goal from the Introduction to evidence and a met / partly met / not met verdict.

VII. DISCUSSION

- What worked: pre-computed graph, authoritative server with filtered views, simplification refactor (about 287 lines removed with all tests green); the backend rewrite into a plain rules core plus a thin Spring service (rules directly unit-testable, eight bugs fixed with regression tests).
- What was hard: map creation effort, disconnect and reconnect behaviour, in-memory state means a restart ends all games.
- Security lesson: role filtering was bypassable, because public player IDs doubled as credentials and the in-memory STOMP broker honoured wildcard subscriptions and client SENDs; fixed with secret tokens, exact-match subscriptions and dropping client SEND frames.
- Trade-offs revisited: no accounts (per-player secret tokens instead), no database, desktop-first layout.
- Threats to validity: small sample, no automated frontend tests, benchmark scope.

VIII. CONCLUSIONS AND FUTURE WORK

- Conclusion: restate aim, what was delivered, key evaluation results.
- Future work framed for a next ENGR489 student:
 - In-game tutorial / onboarding (top user request).
 - Configurable lobbies and rule variants.
 - Persistence (database) and reconnect across restarts; accounts.
 - Mobile layout and touch interaction.
 - Automated map generation from open data.
 - Statistical balance analysis using automated bot players (Mr X versus detective win rates).
 - Larger user study, including more Mr X players.

ACKNOWLEDGMENTS

- Supervisors; course coordinator (ethics approval); user-testing participants; the friend or teammate who helped with the map, if any.

REFERENCES

- [1] A. Author, "Placeholder website title," [Online]. Available: <https://example.com>, 2026, accessed: Sep. 20, 2026.